CampusFind

A lost and found web app for campuses

Project guide: what it is, how it works, and how to run it.

Stack	HTML, CSS, JavaScript, Flask (Python), Supabase
Type	Full-stack web application (college project)
Repository	github.com/h-abeeb/findCampus

1. The problem

Students and staff lose belongings in colleges, offices and public places all the time. There is usually no central place to report a lost item or to find out whether someone has handed one in. People fall back on WhatsApp groups, notice boards and the security desk, which are scattered and hard to search.

Goal: build one system where people can **report**, **search** and **manage** lost and found items.

2. What CampusFind does

- Report a **lost** or **found** item with a title, description, category, location, contact email and an optional photo.
- Browse reports in two sections: **Lost items** and **Found items**.
- Search by keyword across the reports.
- Open an item to see its full details.
- For **found** items, a person can submit an ownership claim describing how they can identify the item.
- For **lost** items, a finder simply contacts the owner using the contact email shown on the item.
- An admin can approve a claim using a secret token.

Why the claim form only appears on found items

A lost-item post comes from the owner, so a finder just needs to reach them. A found-item post comes from someone who has the item, so the real owner must prove it is theirs. That is why only found items have a claim form.

3. Tech stack

Layer	Technology	Role
Frontend	HTML, CSS, vanilla JavaScript	Pages the user sees and clicks. Runs in the browser.
Backend	Python + Flask	A JSON API. Receives requests, validates data, talks to the database.
Database	Supabase (PostgreSQL)	Stores items and claims.
File storage	Supabase Storage	Stores uploaded item photos in a public bucket.

Why Flask? It is small and quick to set up, and it is Python, which also makes it easy to add machine learning later. **Why Supabase?** It gives a hosted PostgreSQL database, file storage and (later) login with a generous free plan.

Why JavaScript at all? Browsers only run JavaScript. Flask runs on the server and can only reply after the browser asks. JavaScript in the page calls the Flask API in the background and updates the page without reloading.

4. How the pieces fit together

```
Browser (HTML/CSS/JS)            Flask backend              Supabase
---------------------            -------------              --------
index.html, report.html   -->    app.py                -->  PostgreSQL tables
item.html                 fetch  items.py, claims.py   API    (items, claims)
api.js (fetch helpers)    <--    returns JSON          <--  Storage (photos)
```

The frontend never talks to the database directly. Every action goes through the Flask API, which holds the secret Supabase key. This keeps the key safe on the server.

Example: reporting an item

- The user fills in **report.html** and clicks Publish.
- If a photo was chosen, JavaScript first sends it to **POST /api/upload**. Flask saves it in Supabase Storage and returns a public URL.
- JavaScript then sends the form data (plus the photo URL) to **POST /api/items**.
- Flask validates the fields, inserts a row into the **items** table, and returns the new item.
- The browser redirects to **item.html?id=...** to show the saved report.

5. Project structure

```
campusFind/
|-- .gitignore
|-- README.md
|-- backend/
|   |-- app.py             # creates the Flask app, CORS, registers routes
|   |-- db.py              # connects to Supabase using .env values
|   |-- items.py           # report, list, search items; upload photos
|   |-- claims.py          # submit a claim; admin approves a claim
|   |-- requirements.txt   # Python packages
|   `-- .env               # secrets (never committed to GitHub)
`-- frontend/
    |-- index.html         # home: search + Lost and Found sections
    |-- report.html        # form to report an item
    |-- item.html          # item details + claim form (found items)
    |-- api.js             # helper functions that call the API
    `-- style.css          # styling
```

File by file

File	What it does
app.py	Starts Flask, allows the frontend to call the API (CORS), and registers the items and claims routes.
db.py	Reads SUPABASE_URL and SUPABASE_KEY from the .env file and creates the database client.
items.py	Handles creating items, listing and searching them, fetching one item, and uploading photos.
claims.py	Saves a claim for an item, and lets an admin approve a claim when the correct token is sent.
api.js	One place for all fetch calls: listItems, getItem, reportItem, uploadImage, submitClaim. Also holds escapeHtml.
index.html	Shows reports in two grids (Lost and Found) and runs the search box.
report.html	The report form. Uploads the photo first, then submits the report.
item.html	Shows one item. Shows the claim form only for found items.
style.css	All colors, layout and component styles.

6. Database

Two tables in Supabase PostgreSQL. A claim belongs to an item.

items

Column	Type	Meaning
id	uuid	Unique id, generated automatically
title, description	text	What the item is
category, location	text	Type of item and where it was lost or found
type	text	Either 'lost' or 'found'
contact	text	Reporter's contact email
image_url	text	Public link to the photo (optional)
status	text	Starts as 'open'
created_at	timestamp	When the report was made

claims

Column	Type	Meaning
id	uuid	Unique id
item_id	uuid	Which item is being claimed
claimant_name, claimant_email	text	Who is claiming
message	text	How they can identify the item
status	text	Starts as 'pending', becomes 'approved'
created_at	timestamp	When the claim was made

7. API endpoints

Method	Endpoint	Purpose
GET	/api/health	Check that the server is running
GET	/api/items?search=	List items, optionally filtered by keyword
GET	/api/items/<id>	Get one item
POST	/api/items	Report a new item
POST	/api/upload	Upload a photo and get its public URL
POST	/api/items/<id>/claims	Submit a claim for an item
POST	/api/claims/<id>/approve	Approve a claim (needs the X-Admin-Token header)

8. Setup and running

What you need

- Python 3 and Git installed.
- A free Supabase account and project.

Step 1: Supabase

- In the SQL Editor, create the **items** and **claims** tables (see section 6).
- In Storage, create a **public** bucket named **item-images**.
- From Project Settings, copy the project URL and the **secret** key.

Step 2: Backend

```
cd campusFind/backend
python -m venv venv
venv\Scripts\activate          (Windows)
source venv/bin/activate        (Linux / Mac / WSL)
pip install -r requirements.txt
```

Create a file named **.env** inside the backend folder:

```
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_KEY=your_secret_key
ADMIN_TOKEN=any_string_you_invent
```

Important: the secret key must stay on the server. Never put it in the frontend and never commit the .env file. ADMIN_TOKEN is not a Supabase value; it is a password you make up for the approve-claim route.

```
python app.py          # API runs at http://127.0.0.1:5000
```

Step 3: Frontend

In a second terminal:

```
cd campusFind/frontend
python -m http.server 8000
```

Open **http://127.0.0.1:8000** in the browser.

9. Key concepts (good for explaining the project)

Concept	Plain explanation
REST API	The backend exposes URLs (endpoints) that return JSON. The frontend calls them with fetch().
CORS	Browsers block a page on one address (localhost:8000) from calling a server on another (127.0.0.1:5000) unless the server allows it. app.py adds the headers that allow it.
Row Level Security (RLS)	A Supabase feature that blocks database access unless a policy allows it. The backend uses the secret key, which bypasses RLS, so it must never reach the browser.
XSS protection	User text is passed through escapeHtml() before being put into the page, so someone cannot inject a script through an item title.
Virtual environment	A private folder of Python packages for one project, so projects do not conflict.
Environment variables (.env)	Secrets live outside the code and are excluded from Git.

10. Existing platforms

Lost-and-found software already exists, but it mostly targets organizations.

Type	Examples	Gap
Enterprise software	iLost, Troov, FoundFastApp, Lost and Found Manager	Priced and built for hotels, airports and venues; staff-run.
Campus projects	HuskyFind (UW), uFindIt (uOttawa), Reclaim (Goucher), Mila Kya? (DTU)	Mostly single-campus projects with limited verification or matching.
Informal channels	WhatsApp groups, notice boards, security desk	Scattered, unsearchable, no verification.

CampusFind's angle: a free, student-first tool with proof-based claim verification, planned automatic matching, and support for any location (campus, office or public place).

11. Current limitations and roadmap

Known limitations

- No login yet, so anyone can post or claim under any email.
- Contact emails are shown publicly on items.
- Claims are saved, but there is no page for reporters to review them.
- Approving a claim does not yet change the item's status.
- Search loads only the latest records and matches keywords, not meaning.
- There is no filter by category or date.

Planned improvements, in order

- **1. Login** with Supabase Auth (college email, magic link or OTP). Browsing stays public.
- **2. Verification:** the finder adds a private question and answer when reporting a found item. The claimant must answer it. Contact details are revealed only after approval.

- **3. Better search:** filters by type, category and date.
- **4. Status flow:** open, claimed, closed, plus a My Items page.
- **5. ML matching:** use a pretrained sentence-embedding model (for example all-MiniLM-L6-v2) and cosine similarity to suggest possible matches between lost and found reports. This is free and runs inside the Flask server. Matches are shown as suggestions with a score, never as confirmed.
- **6. Email notifications** when a possible match or claim arrives.

About the ML plan

The model is pretrained and used through a library, not trained from scratch. The project's own work is the pipeline: turning each report into a vector, comparing lost reports against found reports, choosing a score threshold, and showing the results safely.

12. Quick reference

Task	Command
Start backend	cd backend, activate venv, python app.py
Start frontend	cd frontend, python -m http.server 8000
Open the app	http://127.0.0.1:8000
Push changes	git add . ; git commit -m "message" ; git push